



DATA3001: Data Science and Decisions

NRL-2 Model Report

Analyzing Play-the-Ball Speeds in Rugby League

Aidan Odenthal, Junyi Hao, Kelvin Yu & Sophie Lian

Table of Contents

1 Executive summary	2
2 Background	2
2.1 Introduction	2
2.2 Importance of Play-the-ball	3
2.3 Previous work	3
2.4 Objective	4
3 Scope	4
4 Plan	5
5 Findings	6
5.1 Exploratory Data Analysis & Feature Engineering	6
5.2 Modeling	10
5.3 Conclusion	13
6 Recommendations	14
7 References	16
8 Appendix	16

1 Executive summary

“Play-the-ball” (PTB) speed can be a key factor in deciding game success. Coaches may implement these insights in key decision-making practices and gameplay tactics. This project investigates the main factors which affect the play-the-ball speeds and provides a regression model to predict PTB speeds using data from over 300 NRL games played from 2020 to 2023.

Ultimately, this project’s objectives are two-fold: to identify significant features which determine PTB speed and to develop a regression model to predict play-the-ball speeds, in order to best inform high-level rugby league strategies. This project identified the main contributors towards determining PTB speeds to be the following 13 features:

- the difference between the teams’ scores (**CurrentMargin**)
- elapsed time since the kickoff in the current half (**ElapsedTime**)
- whether a team was at home or away (**IsHome**)
- position of a player (**PositionId**)
- whether the team won or lost a tackle (**PTB.Tackle.Result**)
- whether a player was also a starting player for the match (**RunOn**)
- whether a set was a good ball or a yardage (**Set.Type**)
- the tackle number within the current set (**Tackle**)
- number of effective players involved in the tackle (**Total.Involved.Tacklers**)
- running total of the seconds in possession by either team (**TotalPossessionSecs**)
- weather condition (**WeatherConditionName_Rain**)
- zone column on the field in which the event occurred (**Zone Possession x**)
- zone row on the field in which the event occurred (**Zone Possession y**)

However, of the 13 features, 3 features were found to be the most influential in determining the PTB speeds, which were **PTB.Tackle.Result**, **Set.Type** and **Tackle**.

Furthermore, of the six regression models tested - multilinear, Ridge, Least Absolute Shrinkage and Selection Operator (LASSO), elastic net, random forest and XGBoost, our overall best performing model was XGBoost. Analysis of the Ordinary Least Squares (OLS) summary and residuals exemplified XGBoost’s consistent high performance and strong generalization ability for predicting PTB speeds.

2 Background

2.1 Introduction

In the professional rugby league and rugby union space, game analysis has historically been a cornerstone for enhancing team strategies, performances and real-time decision making. Successful game outcomes can be attributed to the fine and complex web of interactions between players’ technical and physical capabilities. Capturing these intricacies is the central challenge faced by current day empirical game analysts. Now with the increasing availability of modern analytical tools and spanning potential of analytical insights, National Rugby League (NRL) strategy and decision-making practices have seen shifts towards more informed and data-driven approaches.

This project investigates the main factors affecting play-the-ball (PTB) speeds. PTB speeds have been widely claimed by rugby league coaches to be a critical factor in determining the outcome of a game (Anderson, 2000) since the introduction of the rule in the early 1900s. Therefore, we have identified the most influential features in determining PTB speeds and developed a model to predict the PTB speeds which can enable further investigations into the potential relationship between PTB speeds and game outcomes.

2.2 *Importance of Play-the-ball*

“Play-the-ball” (PTB) refers to the rule in rugby league following a tackle, where the referee calls for the tackled player to place the ball on the ground and roll it backward to pass to a teammate, known as the dummy-half. PTB speed is measured as the time taken from the referee’s call to when the ball is played after being rolled through the player’s legs. Faster PTB speeds can significantly affect deciding factors in a game by creating advantages for the team in possession such as increasing try-scoring opportunities, gaining field position advantages, or simply achieving a repeat set to maintain possession. Consequently, as teams increasingly prioritise faster PTB speeds, coaches are also seeking more dynamic ruck plays and faster dummy-half ball reception to expedite PTBs (Smith, 2000).

With additional PTB rules implemented over the following century, PTB speeds are now the product of a complex interaction of features including player attributes, tackle outcomes, tackle location and the number of defensive markers involved. Understanding and predicting PTB speeds can ultimately influence decision-making in player positions, practice goals and gameplay tactics. Analysis of PTB speeds can thus provide valuable insights for rugby league coaches with the potential to better inform and improve NRL strategies.

2.3 *Previous work*

Previous research on the influence of PTB speeds is limited or dated. Yet the significance of PTB’s influence on game dynamics has been repeatedly reemphasized by coaches for decades (e.g. Anderson, Elliott, Smith, 2000) who dispute that “those couple of seconds [during PTB] ... can make a difference in the winning result”. Despite the anecdotal evidence from experienced sport professionals, there is a severe need for statistical evidence to objectively evaluate the true impact of PTB on game outcomes.

To date, only Eaves (2003, 2006, *et.al.* 2007) and Hughes (2003) have examined the effects of PTB speeds on rugby league games. Assessing PTB speeds and other central success factors is particularly challenging. Eaves and Evers (2007) note that the majority of statistical inferences are drawn from differences between full-game reports of the winning and losing teams. This is problematic as game performance is dynamic, meaning that game intensity and critical timings fluctuate all throughout. By solely observing full-game data, many plays which contribute to successful performances are missed. Performative indicators may be inaccurate or over generalized as a result, making it difficult to determine the true factors underlying successful performances. To mitigate these effects, in his 2006 study, Eaves examined the differences in the means of the data from both the full-game reports and during critical periods between winning and losing teams. However, the assumption that teams should strive for faster PTB speeds as it leads to greater win rates is not necessarily accurate. Players may occasionally employ slower PTB speeds as it provides the offensive team time to better prepare “for big plays” or to minimize the risk for player errors (Smith, 2000). Therefore, comparing the differences between winning and losing teams may not yield definitive evidence of a relationship between PTB speeds and game outcome.

Hughes' *et.al.* 2001 study introduced an alternative method to assessing performative indicators without utilizing full-game reports. The study aimed to identify factors which destabilize the flow of a game, and these pivotal periods were referred to as perturbations. These perturbations were believed to catalyze critical incidences or opportunities to score. Eaves and Evers (2007) proposed employing this approach using pattern analysis preceding perturbations to investigate the impact of perturbations on game outcome. Subsequently, elements which contribute to a perturbation such as PTB speeds can thus be examined. Their findings suggest a minimal relationship between PTB speeds and the actions which lead to critical incidences. On the contrary, the authors acknowledge that their conclusion on the significance of PTB speeds, which are based solely on whether or not a perturbation occurred, is a possible oversimplification of the interaction between PTB speeds and critical incidence.

As the NRL has undergone a number of rule changes pertaining to PTB and subsequent gameplay changes since many of these studies, the relevance and extent of their conclusions must be reassessed.

2.4 Objective

This project aims to identify the main factors which affect the PTB speeds. We will also attempt to build a model to predict PTB speeds to provide further support for our findings. Analysis of the findings will reveal the most significant features which contribute to faster play-the-ball speeds. NRL coaches can therefore employ these insights in coaching strategies, redefine practice goals and enhance gameplay tactics to achieve better performances.

3 Scope

The primary deliverables of our finalised project includes an extensive data modeling report that details the relationship between our predictors and the target variable of PTB speeds. The modeling report also provides the key insights that we have derived from our model that highlights the decisive factors that influence PTB speeds, alongside actionable recommendations that NRL coaches can effectively utilize based upon the data. In addition, our report includes our Python source code that clearly outlines the essential processes such as data cleaning, feature engineering, modeling, and evaluation, ensuring easy reproducibility for future use. In collaboration, we will also deliver an oral presentation to our stakeholders that aims to convey key aspects of our project. This presentation will be accompanied by a slide deck to provide visual support and enhance understanding.

Our team has been provided with the following information to work with: a data set that has been cleaned and modified by peers, ready for modeling. The dataset is a collection of over 300 NRL games played between 2020-2023, comprising of 36 features and 112,027 observations related to PTB speed. Accompanying this is a README file which outlines the problem statement: *'What are the key determinants of play-the-ball speed, and how might teams adjust their strategies to optimise it?'*. The README file also provides the definition and descriptions of each variable used as well as feature engineering and data cleaning strategies applied to the original dataset.

Multiple key assumptions were made for this project. First of all, we assume that the given dataset accurately and fully represents the range of conditions and factors that occur within a real game which influences PTB speed, ensuring

that the data is unbiased. Furthermore, we also assume independence of observations meaning that each observation in the dataset is independent of one another which is vital for our model's validity. We also assume that external factors, such as rule changes or shifts in playing styles, remain consistent between 2020 and 2023 since differences could affect the relationship between the predictors and PTB speed. Finally, our goal is not only to maximise model accuracy but also its interpretability so that we can effectively answer our problem statement. Therefore we may prioritize interpretability over predictive power if necessary, to offer clear and actionable insights and recommendations for coaches.

For this project to be successful, several key criteria must be met. First of all, our chosen model should achieve a high level of accuracy while being interpretable. Since we have chosen the XGBoost regression model, one of the primary metrics to determine and compare accuracy is via the Mean Squared Error (MSE). To ensure success, the MSE should be relatively low compared to the range of our target variable of 5.44 units (from 0.76 to 6.20). This indicates that our model is performing well and can accurately and reliably capture the relationships within the data. Furthermore, comparing the MSE of our model to a baseline model, such as a multi-linear regression will serve to reaffirm its effectiveness. Besides accuracy, our model should also have clear interpretability by being able to identify the most significant predictors of PTB speed as well as illustrating the magnitude of their impact. The project is successful if we can deliver clear, meaningful insights into the factors influencing PTB speed and utilize this to develop practical recommendations for coaches to enrich their strategies or improve players' skillsets.

Last of all, the documentation within our report should be clear and comprehensive. This includes detailed explanation of our procedures such as data cleaning, feature engineering and model development with its justifications. Most importantly, the report should be able to clearly explain how our recommendation can be effectively interpreted and implemented in real-world settings.

4 Plan

We began the project by gaining a clear understanding of the context and goals. This was accomplished through reviewing the README file and conducting external research, such as general domain knowledge of NRL and previous related work.

However, we encountered complications when we reviewed our dataset such as similarities between the predictors of PTB Tackle Result and PTB Contest leading us to examine the potential to keep both. Additionally, we unfortunately lacked some domain knowledge such as the difference between a good ball and a yardage. These were addressed by our sponsor during a conference in the second week of the project.

Our next step then was proceeding with exploratory data analysis (EDA) where our plan included the following: cleaning the dataset by removing missing values, outliers and conducting feature selection. Due to the large number of features, we conducted feature selection by employing correlation matrices and Variance Inflation Factor (VIF) to assess multicollinearity whilst using graphs and domain knowledge to handle outliers. Furthermore, final feature significance was determined by evaluating its impact on multiple regression models including OLS, Lasso, Ridge and

ElasticNet. This was successful as it helped justify the removal of 27 predictors in the dataset since the performance change was minimal.

We planned to develop and compare a number of models with a range of complexities, using model performance metrics such as the mean squared error (MSE), root mean squared error (RMSE) and R-squared value.

We noted that the PTB speed model is more of a static regression problem that aims to predict PTB speeds based on input characteristics in a specific scenario (e.g., player position, weather, stage of play, etc.). PTB velocity is a continuous variable (velocity in time), so we needed a regression model that can handle the output of continuous values. Since the target variable is continuous, we can exclude classification models such as decision tree classification, SVM for classification, etc. These models are suitable for classification tasks, but not for continuous value prediction. Instead, we used regularization techniques and ensemble learning methods via bagging and boosting which would also improve the model's predictive accuracy. These included a simple multilinear regression, regularized linear regression models such as LASSO, Ridge and ElasticNet as well as some higher complexity models such as XGBoost and random forests. The regularization methods helped address overfitting, reduce variance and shrink insignificant coefficients. In addition, XGBoost, a boosting method, targets bias reduction and achieves relatively high predictive accuracy whilst simultaneously incorporating regularization techniques. On the contrary, random forests, a bagging method, was selected to decrease variance whilst enhancing predictive performance via ensemble learning. Parameter tuning is applied across all models, where possible, to ensure that the optimal version of each model is achieved. This helps to fine-tune the model's performance and maximize its predictive accuracy.

After evaluating the models via performance metrics, we selected the XGBoost as the best model that balances high predictive accuracy with the ability to be easily interpretable.

Finally, based on our problem statement, we then identified the most significant factors influencing 'PTB speed' and derived practical recommendations based upon these findings.

5 Findings

All code, visualizations, graphs and other output can be found in the Appendix (Section 8).

5.1 *Exploratory Data Analysis & Feature Engineering*

Exploratory Data Analysis (EDA) was performed to assess and validate the quality of the dataset. This process involved cleaning the dataset of missing values, outliers as well as conducting feature selection. Furthermore, EDA also served to guide our modeling approach by deepening our understanding of the dataset and hence allowing us to hypothesize possible relationships within it.

As a part of the early consistency and quality checks, we initiated the process with data cleaning. During our checks for possible missing values in the dataset, we identified that one of the features, **PTBDefence**, was mostly empty (87,992 missing values out of the total 112,027). Since this binary feature represents whether or not the defence was set, filling in these missing values could introduce bias into our results. Thus, PTBDefence was removed.

Furthermore, any other rows containing missing observations were similarly removed to maintain the integrity of the data.

Rows (y)	2	3	4	5	6	7	8	9	10	11
7	3.97	3.82	3.62	3.56	3.65	3.57	3.55	3.59	3.79	3.96
6	3.82	3.60	3.50	3.45	3.42	3.48	3.51	3.61	3.73	3.88
5	3.83	3.54	3.43	3.36	3.33	3.36	3.45	3.57	3.66	3.77
4	3.86	3.52	3.40	3.33	3.29	3.29	3.38	3.50	3.53	3.72
3	3.79	3.53	3.47	3.37	3.30	3.34	3.38	3.50	3.55	3.76
2	3.86	3.54	3.48	3.40	3.42	3.44	3.49	3.63	3.73	3.89
1	3.96	3.72	3.61	3.58	3.57	3.72	3.69	3.77	3.79	3.97

Additionally, we removed a number of variables which were considered unrelated or redundant towards predicting PTB speeds. For instance, the feature **Score** is fully mapped by **CurrentMargin** and can be derived using the conditions `CurrentMargin>=0` or `CurrentMargin<0`. Similar calculations and mappings can be obtained for **GameTime**, **Half** using `ElapsedTime` and **OppPossessionSecs** using the difference between `TotalPossession` and `PossessionSecs`. **AwayScore** and **OppScore** could also be determined from the sum of `HomeScore` and `CurrentMargin`. The features **ClubID** and **OppositionID** are categorical variables which are both used to differentiate between the attacking and defending teams. As they provided similar information, **ClubID** was removed and **OppositionID** was kept as a dummy variable. On the other hand, other features could be inferred from existing features. This included **PTBContest** which could be inferred from **PTBTackleResult** and likewise for **ZonePhysical** from **ZonePossession**. **EventName** contained the name of the event in which the game was played and had minimal predictive significance so was dropped. Hence, these variables provided little additional predictive value and were dropped to improve model performance.

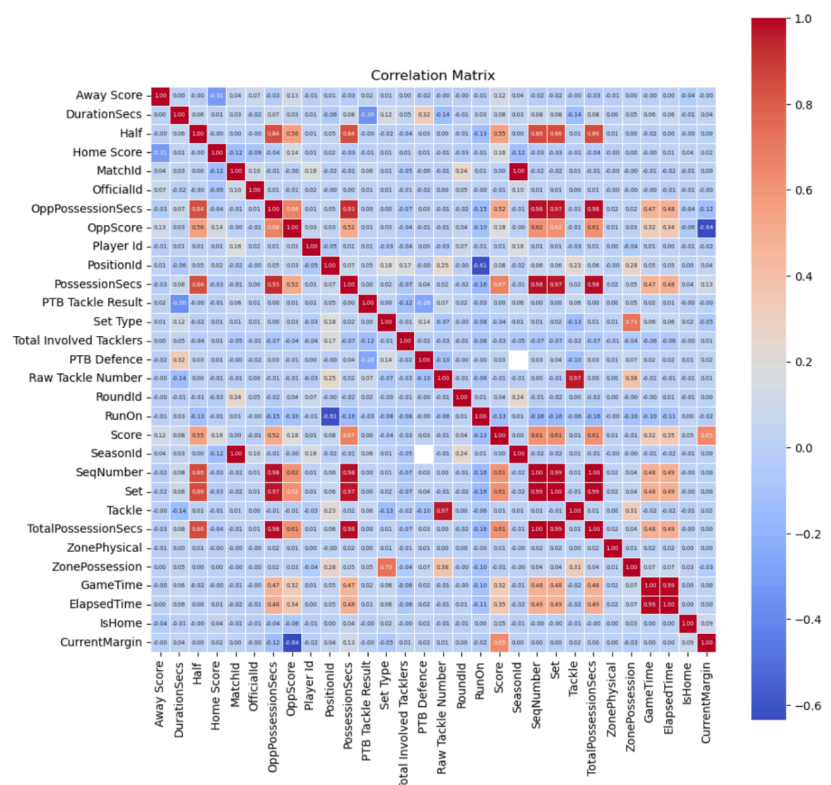


Figure 5.11 Heatmap Correlation Matrix of all Features

Analysis from the correlation matrix heatmap (Figure 5.11), revealed several pairs of highly correlated variables (Correlation>0.7). Notably, these pairs included **MatchId** and **RoundID**, **PossessionSecs** and **TotalPossessionSecs**, and **SetType** and **ZonePossession**.

Ultimately, we dropped **PossessionSecs** as **TotalPossessionSecs** would provide a more complete view of team possession statistics and offer deeper insights into team fatigue and game control. In addition, **RoundID** was dropped since **MatchID** could capture more game information such as the playing team. Finally, **SetType** was also removed in favour of **ZonePossession**, which likely provided a more comprehensive measure of the state of the game.

Part of our EDA process also included analysis of the Variance Inflation Factor (VIF) of all the features. VIF was employed to reduce multicollinearity in our dataset by identifying highly correlated features and removing those with excessively high VIF values, as seen in Figure 5.12 (see appendix for enlarged results).

Index	Feature	Variance Inflation Factor (VIF)
0	Away Score	2.011938
1	DurationSecs	1.253010
2	Half	81.930792
3	Home Score	2.104919
4	MatchId	622.527183
5	OfficialId	1.056650
6	OppPossessionSecs	inf
7	OppScore	inf
8	Player Id	1.012881
9	PositionId	1.853186
10	PossessionSecs	inf
11	PTB Tackle Result	1.196111
12	Set Type	3.117679
13	Total Involved Tacklers	1.071384
14	Raw Tackle Number	15.490458
15	RoundId	625.068105
16	RunOn	1.647206
17	Score	inf
18	SeasonId	0.000000
19	SeqNumber	329.357503
20	Set	82.178051
21	Tackle	14.228941
22	TotalPossessionSecs	inf
23	ZonePhysical	1.002946
24	ZonePossession	3.732256
25	GameTime	86.912625
26	ElapsedTime	68.128301
27	IsHome	1.103638
28	CurrentMargin	inf

Figure 5.12 VIF results for all features

Index	Feature	Variance Inflation Factor (VIF)
0	Away Score	2.011938
1	DurationSecs	1.253010
3	Home Score	2.104919
5	OfficialId	1.056650
8	Player Id	1.012881
9	PositionId	1.853186
11	PTB Tackle Result	1.196111
12	Set Type	3.117679
13	Total Involved Tacklers	1.071384
16	RunOn	1.647206
18	SeasonId	0.000000
23	ZonePhysical	1.002946
24	ZonePossession	3.732256
27	IsHome	1.103638

Figure 5.13 VIF results after removal of high VIF features

Model	MSE _{train}	MSE _{test}	RMSE _{train}	RMSE _{test}	R ² _{train}	R ² _{test}	AIC	BIC
Linear Regression	0.4027	0.4109	0.6346	0.6410	0.1404	0.1357	160023.3717	160405.7345
Ridge Regression	0.4027	0.4109	0.6346	0.6410	0.1404	0.1357	160023.3717	160405.7345
LASSO Regression	0.4551	0.4629	0.6746	0.6804	0.0285	0.0263	160023.3717	160405.7345
ElasticNet Regression	0.4246	0.4323	0.6516	0.6575	0.0936	0.0907	160023.3717	160405.7345
XGBoost Regression	0.3872	0.3986	0.6223	0.6133	0.1734	0.1617	-	-

Figure 5.14 Summary of select Models without Interaction terms

Model	MSE _{train}	MSE _{test}	RMSE _{train}	RMSE _{test}	R ² _{train}	R ² _{test}	AIC	BIC
Linear Regression	0.4300	0.4418	0.6557	0.6647	0.1255	0.1233	165407.5763	165566.1169
Ridge Regression	0.4300	0.4418	0.6557	0.6647	0.1255	0.1233	165407.5763	165566.1169
LASSO Regression	0.4811	0.4936	0.6936	0.7026	0.0215	0.0206	165407.5763	165566.1169
ElasticNet Regression	0.4480	0.4604	0.6693	0.6785	0.0889	0.0865	165407.5763	165566.1169
XGBoost Regression	0.4162	0.4320	0.6451	0.6573	0.1535	0.1428	-	-

Figure 5.15 Summary of select Models with Interaction terms

Note that the features with infinite VIF values have been removed in earlier EDA procedures. In addition, features with a VIF of 0, such as **SeasonID**, serve as unique identifiers and do not exhibit a linear relationship with other variables.

RawTackleNumber was dropped as it exhibited a high VIF value of 15.49. It was chosen to be removed over **Tackle**, as its removal lowered train and test MSE scores although both variables indicate tackle number. In addition, other features such as **ZonePossession** could capture similar game information as the two features so we deemed that they did not provide enough meaningful contribution to the dataset. Moreover, **SeqNumber** produced an extremely high VIF value of 329.36. This suggests that there is high multicollinearity with **SeqNumber** and did not appear to provide

meaningful value towards predicting PTB speed so it was removed. The feature **Set** was also dropped due to its high VIF value of 82.18. Since **ElapsedTime** was a more accurate measure of team fatigue, we deemed **Set** insignificant. Figure 5.13 (see appendix for enlarged results) displays the updated VIF values of the remaining features after the removal of those with higher VIFs. As anticipated, the VIF values have contracted significantly. Since the remaining VIFs < 5, this is a clear indication that multicollinearity has been effectively reduced and the remaining features exhibit low correlation with one another.

We also investigated the impact of interaction terms in our exploratory data analysis. Using the correlation matrix and our domain knowledge, we tested the interactions among **ElapsedTime** and **CurrentMargin**, **Total Involved Tacklers > 2** and **PTBTackleResult**, and **SetType** and **PTBTackle Result**.

For the interaction between **ElapsedTime** and **CurrentMargin**, we hypothesized that as the game nears full-time, the winning team tends to play the ball more slowly to minimize the time the ball remains in play. On the contrary, we expected the losing team to play the ball more quickly to maximise scoring opportunities.

The second interaction involving **Total Involved Tacklers > 2** and **PTBTackleResult**, was based on our expectation that an increased number of tacklers would have a greater likelihood of winning a play the ball. This suggests that the attacking player was more prone to be tackled to the ground, resulting in a longer PTB duration.

Moreover, the interaction between **SetType** and **PTBTackle Result** stemmed from our expectation that when there is an attacking set and the attacking player remains standing, the PTB is expected to be much faster.

Figure 5.16 (shown below) highlights extremely low p-values, suggesting that the interaction terms had high statistical significance. However, analysis of the model summary results indicated that the model containing the interaction terms saw worse performance compared to the simple model. As shown in Figures 5.14 and 5.15 (in appendix), there were minimal variations in the MSE, Root Mean Squared Error (RMSE) and R-squared values. Similarly the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) values differed only marginally. Therefore, we decided to omit the interaction terms as they contributed little performative improvements as well as to preserve model simplicity.

Features which did not contribute significantly to the model performance were also removed. By comparing the Mean Squared Error (MSE) of the model before and after these features were removed (shown in Figures 5.14 and 5.21 in appendix), we saw minimal differences. These features were **Anonymize.1PlayerId**, **GameTime**, **Half**, **OppPossessionSecs**, **OppScore**, **PTB Contest**, **Score**, **ZonePhysical**.

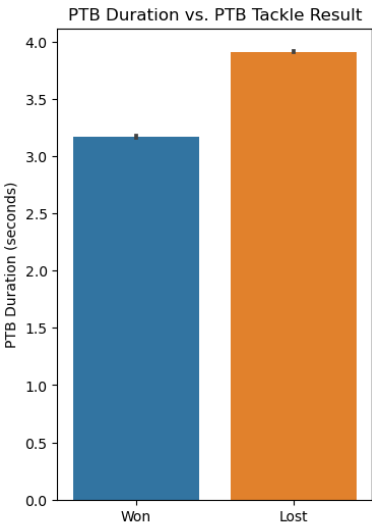
Interaction Term	p-value
ElapsedTime * CurrentMargin	1.536940e-05
Total Involved Tacklers > 2 * PTBTackleResult	4.34553e-30
SetType * PTBTackle Result	5.993324e-09

Figure 5.16 p-values of Interaction Terms

Feature	LASSO Coefficient
PTB Tackle Result	-0.271179
Set Type	0.025810
Tackle	-0.037922

Figure 5.17 Output of LASSO coefficients

Finally, a LASSO regression model was developed to further justify the removal of insignificant features. From the output shown above in Figure 5.17, majority of the feature coefficients have been reduced to zero, with the exemption of three variables: **PTB Tackle Result**, **Set Type** and **Tackle**. Therefore, these features are likely to be the most critical determinants underlying PTB speeds. This conclusion also aligns with our EDA results, as seen in the PTB Duration vs PTB Tackle Result bar graph which



highlights the significant drop in PTB speeds when a tackle is won by the offensive team.

In summary, the following features were removed: Anonymize.1PlayerId, ClubID, GameTime, Half, OppScore, PTB.Contest, PTB.Ultimate.Outcome, PTBDefence, RawTackle, RoundId, Score, SeasonId, SeqNumber, Set, ZonePhysical.

5.2 Modeling

From research of different modeling techniques in Section 4: Plan, we decided to initially build a number of simple and regularization models, consisting of a linear, ridge, LASSO and elastic net regression. Furthermore, we also developed two complex models, one utilizing bagging methods - Random Forests; and the other using an extreme gradient boosting method - XGBoost.

All the data was split into training 80% and testing sets 20% for model evaluation.

We first attempted to develop models prior to feature selection to assess the impacts of filtering certain features.

Subsequently, we tested all the model performances with the features selected for filtering from Section 5.1 EDA.

Analysis of the results revealed that there were minimal changes in the performance of the models, thus we decided to remove the selected features to maintain model simplicity and interpretability. Additionally, from Section 5.1, we investigated the effects of interaction terms in each of the models and concluded minimal performance improvements from the interaction terms, so they were omitted.

Model	(Train)	(Test)	(Train)	(Test)	(Train)	R ² (Test)	AIC	BIC	Remarks
Linear Regression	0.408803	0.415930	0.639377	0.644926	0.127338	0.125192	161217.681656	161348.244549	Baseline model. Similar results to Ridge Regression.
Ridge Regression	0.408803	0.415930	0.639377	0.644926	0.127338	0.125192	161217.681656	161348.244549	Chosen for optimization due to handling multicollinearity effectively.
LASSO Regression	0.458903	0.466365	0.677424	0.682909	0.020391	0.019114	161217.681656	161348.244549	Underperformed with higher MSE and lower R ² values than baseline.
ElasticNet Regression	0.426085	0.433437	0.652752	0.658359	0.090447	0.088370	161217.681656	161348.244549	Worse performance compared to Linear and Ridge Regression.
XGBoost Regression	0.395280	0.406285	0.628713	0.637405	0.156205	0.145478	N/A	N/A	Best overall model with lowest MSE and highest R ² values.

The above results table highlights that XGBoost is the best model, followed by ridge regression (note that the linear model is similar to ridge regression, but ridge regression handles multicollinearity and overfitting better so it was chosen instead), so we intend to further optimize the ridge regression model, the XGBoost model and also decided to evaluate the performance of a random forest model.

Evaluation of the Ridge Regression Model

Out of the multilinear and regularization regression models, the ridge regression presented the best overall OLS results. Therefore, further investigation was conducted on the ridge regression model to evaluate its performance. In particular, analysis of the impact of different lambda values on the ridge regression showed minimal differences in the AIC, BIC and Likelihood Ratio (LR) values across tested different lambda values. Figure 5.22 (in appendix) visualizes the plotted values for the AIC, BIC and LR displayed nearly flat trends, suggesting the model's relative insensitivity to regularization. Among all tested values, $\lambda = 0.01$ yielded the lowest AIC, BIC and LR values. Therefore, a less strict regularization parameter of $\lambda = 0.01$ was selected for the ridge regression model.

Metric	Average Train MSE	Average Test MSE	Train MSE Std Dev	Test MSE Std Dev	Final Train MSE	Final Test MSE
Value	0.4028	0.4018	0.00047	0.00889	0.4041	0.3784

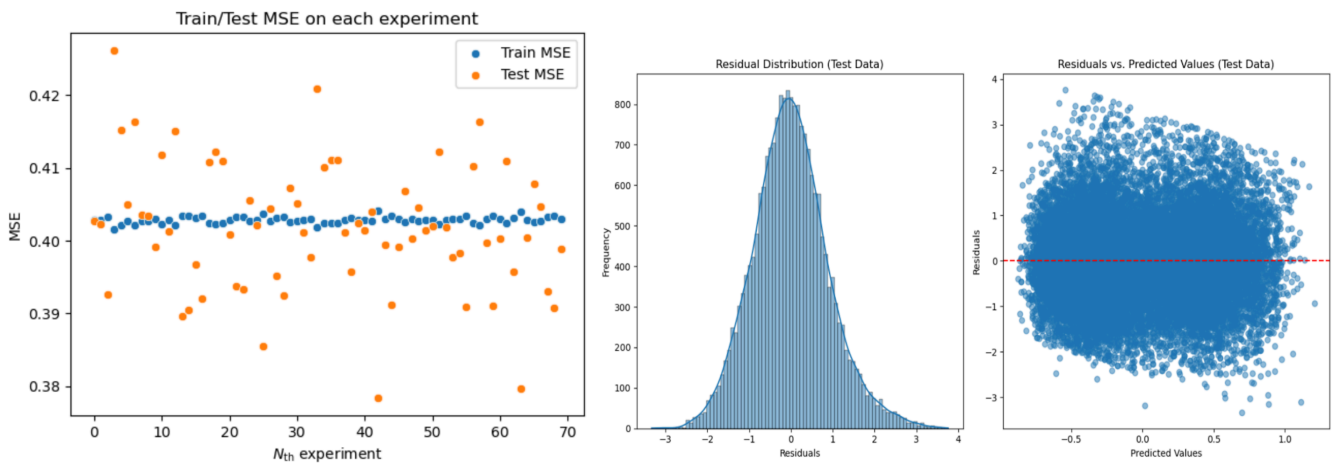


Figure 5.23 Train and Test MSE of Ridge Regression over 70 runs

Ridge Regression

The Ridge regression model also produced similar train and test MSE values ($MSE = 0.403$), which was close to the train and test MSE values of the multilinear regression. The similarity between the MSE values suggests that there is little to no overfitting as the model generalizes well to new data and has stable performance even on the test set. Figure 5.23 further supports these findings as the results reveal a relatively low standard deviation in the MSE values, which reinforces the model's stability across different data splits. In addition, the scatterplot of the test MSE values in Figure 5.23 demonstrate random variance, reflecting low bias and strong model performance.

Overall, the ridge regression model exhibits robust performance, with no clear signs of overfitting or underfitting and a strong generalization ability.

Evaluation of the XGBoost Model

While the Ridge regression model was a consistently well-performing simple model, we decided to also develop a more complex model for greater predictive power. From the earlier model results, the best performing model was the XGBoost which is a gradient-boosting method that can effectively handle more complex relationships in the data whilst reducing the risk of overfitting. This was the justification to optimise the XGBoost model to enhance its performance against the current ridge regression model.

One method of optimisation involves parameter tuning using the RandomizedSearchCV method, which randomly selects combinations from the defined parameter space and evaluates the effect of each combination using 5-fold

cross-validation. Then, the optimal hyperparameter combinations are selected based on the lowest MSE value. We tested 10 random combinations in each run and were able to obtain the optimal parameter combination :

```
{'subsample': 0.8, 'n_estimators': 100, 'min_child_weight': 5, 'max_depth': 5, 'learning_rate': 0.1, 'colsample_bytree': 1.0}
```

This choice of hyperparameters can significantly assist in improving the generalization ability and performance of the XGBoost model.

Metric	Average Train MSE	Average Test MSE	Train MSE Std Dev	Test MSE Std Dev	Final Train MSE	Final Test MSE
Value	0.3835	0.3928	0.00051	0.00884	0.3851	0.3694

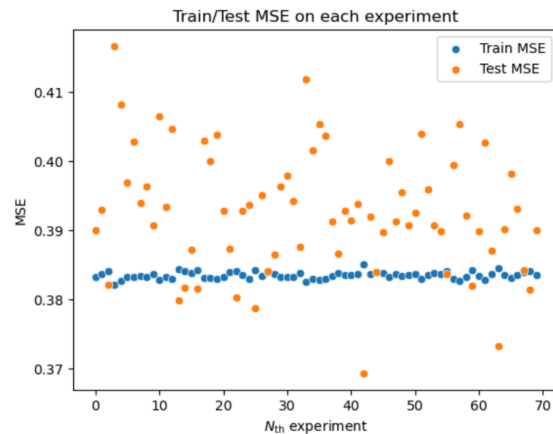


Figure 5.25 Train and Test MSE of XGboost over 70 runs

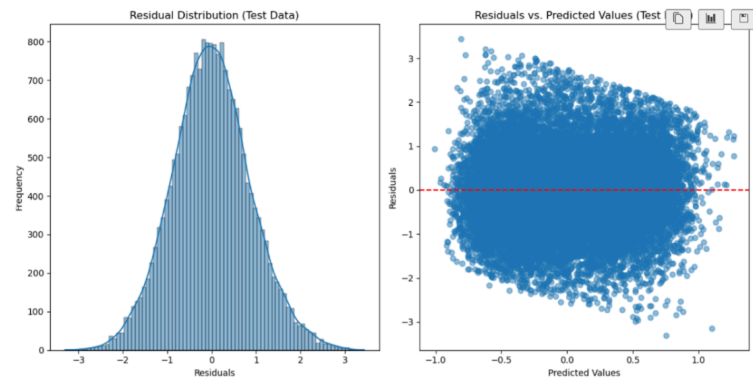


Figure 5.26 Residuals of XGBoost

Using the previously determined optimal set of hyperparameters, the XGBoost model yielded very low test and train MSE values. The test and train MSE values are very close (average $MSE_{test} = 0.383$ and average $MSE_{train} = 0.393$ to 3 d.p., output from Figure 5.25). The small margin between these MSE values is indicative of stable performance and high generalisability. The low standard deviation of the MSE values further solidify the model's consistent performance. Moreover, the scatterplot of the MSE values demonstrate random errors, implying that the model preserves homoskedasticity.

Analysis of the residuals was also conducted to observe its distribution and any potential systematic bias. From Figure 5.26, the residual histogram (shown on the left) displays an approximate normal distribution. This suggests that the prediction errors are distributed symmetrically and that the model errors have little systematic bias, which is an indicator of good model performance. The scatterplot (shown on the right) of Figure 5.26 shows an even distribution of the predicted values on either side of zero, reinforcing uniform distribution of the prediction errors and suggesting no significant systematic trends nor bias.

Overall, the XGBoost model exhibits high predictive power, low bias and variance, and consistent performance over multiple runs.

Evaluation of the Random Forest Model

Metric	Average Train MSE	Average Test MSE	Train MSE Std Dev	Test MSE Std Dev	Final Train MSE	Final Test MSE
Value	0.0587	0.4181	0.00015	0.00985	0.0587	0.4311

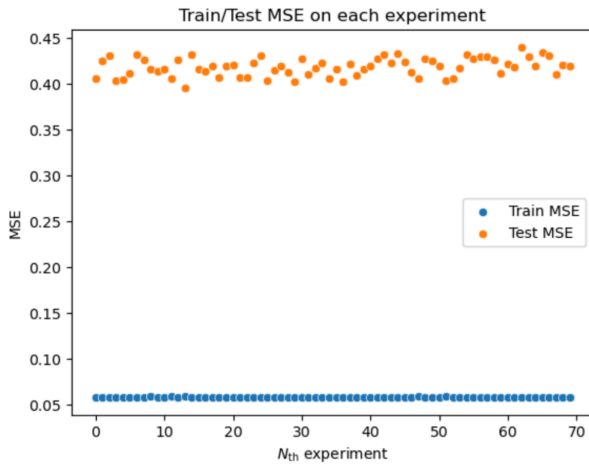


Figure 5.27 Train and Test MSE of Random Forest over 70 runs

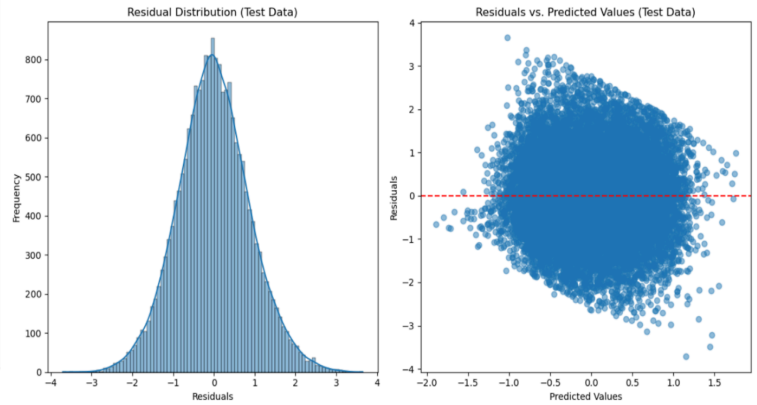


Figure 5.28 Residuals of Random Forest

As detailed earlier, the random forest model demonstrated a significantly lower train MSE value than other models. However, when iterated over 70 runs, Figure 5.27 highlights that there is a large difference between the train and test MSE, implying that the model may be overfitting the data.

Although the residuals suggest some extent of good performance, with little systematic bias in the random forest model, the considerable margin between its MSE values more strongly indicate otherwise. This sizable difference signifies that the model is overfitting during the training stages, leading to the poor generalisation ability during the testing phase. Therefore, we concluded that the random forest model lacked sufficient generalisability and was thus not suitable for our project objective.

5.3 Conclusion

The table below summarizes our modeling results and the examined advantages and disadvantages of Ridge Regression, XGBoost, and Random Forest:

Model	Train MSE	Test MSE	Train R ²	Test R ²	Advantages	Disadvantages
Ridge Regression	~0.409	~0.416	Moderate (~0.63)	Moderate (~0.64)	Handles multicollinearity well, robust against overfitting, consistent performance across splits.	Limited ability to model complex non-linear relationships.
XGBoost	~0.386	~0.405	Low (~0.18)	Moderate (~0.15)	Captures complex patterns, robust against overfitting, low bias, consistent performance.	More complex to tune and implement compared to simpler models.
Random Forest	~0.058	~0.431	High (~0.87)	Low (~0.10)	Captures non-linear relationships well, strong fit to training data.	Severe overfitting, poor generalization to unseen data.

Therefore, for model selection, we chose XGBoost because of its strong ability to capture complex data patterns, offering both high accuracy and generalizability. Notably, Ridge Regression serves as a simpler, more interpretable alternative. However, it suffers from lower accuracy and predictive power compared to XGBoost.

Important features

The main factors which determine PTB speeds were identified to be these **13 features**: CurrentMargin, ElapsedTime, IsHome, PositionId, PTB.Tackle.Result, RunOn, Set.Type, Tackle, Total.Involved.Tacklers, TotalPossessionSecs, WeatherConditionName_Rain and Zone Possession x, Zone Possession y. This was determined by the exploratory data analysis, feature filtering and reinforced by the results from **XGBoost** coefficient shrinkage shown in Figure 5.26.

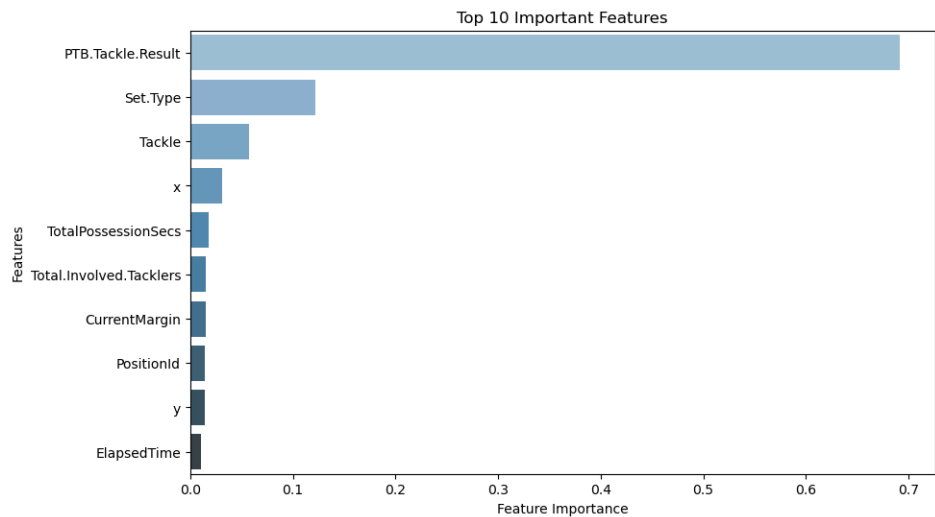


Figure 5.26 Feature Importance Ranked by XGBoost Coefficients

Of the 13 features, analysis of the **XGBoost** results found the most significant features to be **PTB.Tackle.Result**, **Set.Type** and **Tackle**. Thus, the main variables which contribute to determining play-the-ball speeds are whether a tackle was won or lost, whether a set was a good ball or a yardage and finally the tackle number of a set.

6 Recommendations

From our findings, the most critical factors that affect play-the-ball speed were whether a team won or lost the tackle, whether the set was an attacking set or a yardage set, and the tackle number for the set. Thus, we promote the following recommendations for coaches and those interested in enhancing their knowledge of PTB speeds.

Firstly, if the attacking team wins a tackle, they are more likely to achieve faster PTB speeds. From a defensive standpoint, coaches should focus on improving tackling techniques to ensure that attacking players are brought to the ground more effectively and often to slow down PTB speeds. Teams may choose to hone in on strength training, wrapping up the ball to prevent offloads, practicing leg tackles, and tackling in numbers to increase the likelihood of performing a dominant tackle which allows defenders to stay in the tackle for longer to slow PTB speed.

Since attacking sets provide the team in possession a field position advantage, it is crucial for the offensive team to prioritise fast PTBs. Coaches may consider incorporating ruck plays such as short-side attacks or dummy-half scoots to take advantage of quick PTBs. Potential outcomes of these plays include increased set distances, consecutive fast

PTBs, repeat sets, and try-scoring opportunities - all of which are likely to influence the outcome of the game. On the contrary, when teams are focused on gaining ground during a yardage set, then slower PTB speeds can be beneficial as it maximises the time for recovery and decision-making, systematically reducing player error.

Moreover, the tackle number in a set displayed an inverse relationship with PTB speed, with earlier tackles generally correlating with faster PTB speeds. Fatigued players are prone to tackle inefficiency, slow kick chases, and struggle to fight off defenders when running the ball into the defensive line, becoming liabilities in offense and defense by slowing PTBs in attack and hastening them in defense. Specialised endurance training for fatiguing players may help mitigate these effects, and coaches may experiment with tactical interchanges in critical stages of the game to maintain attacking momentum and place defensive pressure on the team in possession, especially during the later tackles of a set.

We hypothesize that some variables not included in the model may be stronger predictors of PTB speed. Based on our domain knowledge, other potentially influential variables include whether or not a line break occurred in the set, the set distance, as well as the average minutes of playtime for each player per season. Furthermore, we believe that PTB Defence was likely a key determinant of PTB speed, as it is likely that players will attempt to play the ball quickly against broken defense. Unfortunately, PTB Defence was excluded due to missing data. This was a major weakness for our model as we anticipate that incorporating this data could potentially improve our model's ability to capture the underlying relationship with PTB speed. Therefore, if more complete data was collected on PTB Defence in the future, a more robust and accurate predictive model could be developed. Alternatively, future analysts may choose to investigate data imputation or augmentation techniques to address the missing data.

In addition, some of the models explored in this project could be further investigated. In particular, the random forest demonstrated some potentially very high predictive power but suffered from limited generalizability. This issue could be amended by further optimizations of the random forest hyperparameters, rigorous feature engineering or utilizing a larger dataset. Yet, the interpretability of our results and current XGBoost model are a key strength in our methodology and these procedures may be a tradeoff of model simplicity and interpretability.

Although our model selection process weighed upon hyperparameter tuning, OLS results and model residuals, future analysts may also choose to utilise k-fold cross validation to further justify model selection as well as gain greater insights into model performance. Future studies may also examine potential non-linear relationships between the predictors and PTB speeds using more advanced techniques such as Principal Component Analysis (PCA) which can handle dimensionality reduction, interaction effects, multicollinearity and preserve variance.

7 References

- Anderson, D. (2000). Coach Talk. Rugby League Coaching Manuals, Book 2. RLCM Coaching Books.
- Eaves, S. J. (2006). 'The convergence of the twain': A notational analysis of northern hemisphere rugby league and rugby union football 1988-2002. PhD Thesis, University of Wales.
- Eaves, S. J. and Evers, L. A (2007) The relationship between the 'play the ball' time, post-ruck action and the occurrence of perturbations in professional rugby league football, International Journal of Performance Analysis in Sport, 7:3, 18-25, DOI: 10.1080/24748668.2007.11868406
- Eaves, S. and Hughes, M. (2003). Patterns of play in international rugby union teams before and after the introduction of professional status. International Journal Of Performance Analysis in Sport. 3(2), 103-111.
- Elliott, M. (2000). Coach Talk. Rugby League Coaching Manuals, Book 2. RLCM Coaching Books.
- Smith, T. (2000). Coach Talk. Rugby League Coaching Manuals, Book 2. RLCM Coaching Books.

8 Appendix

Visualizations, graphs and figures

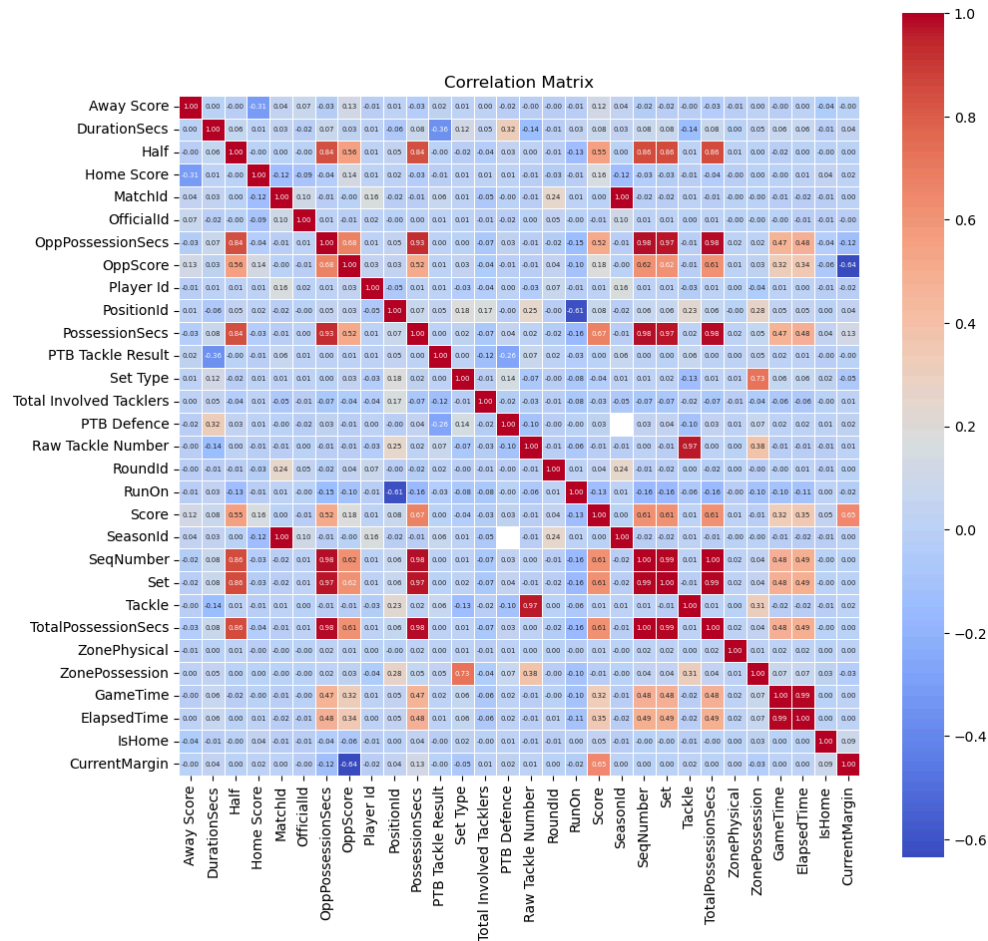
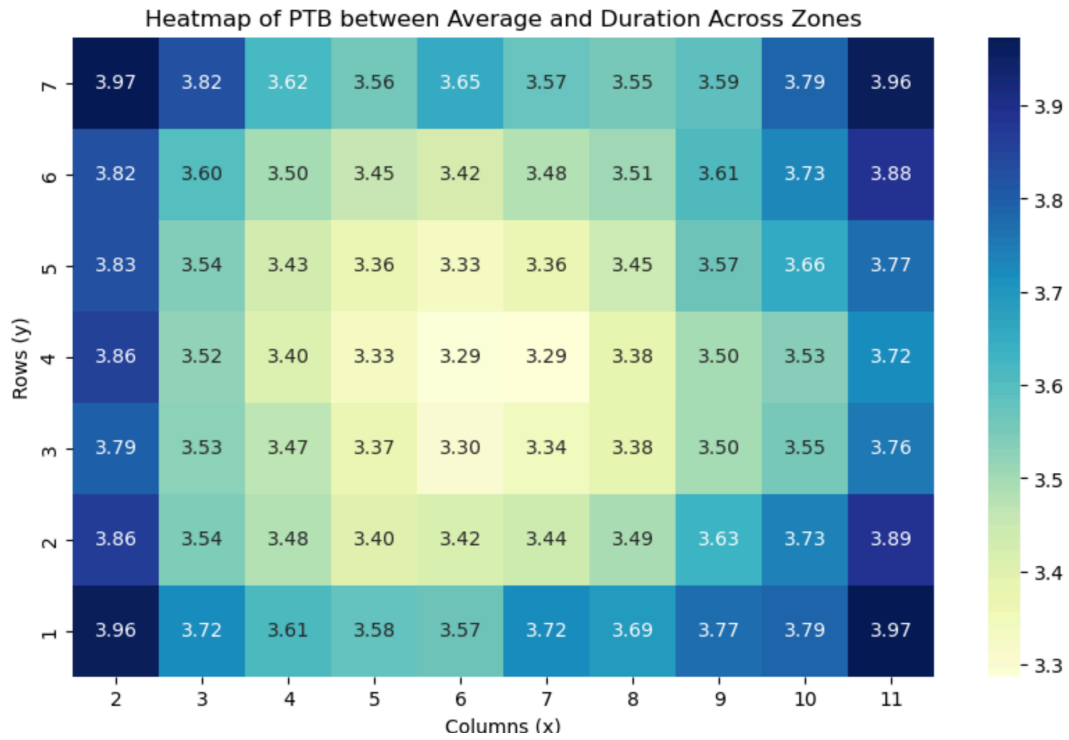


Figure 5.11 Heatmap Correlation Matrix of all Features



Heatmap showing PTB speeds across different zones

Index	Feature	Variance Inflation Factor (VIF)
0	Away Score	2.011938
1	DurationSecs	1.253010
2	Half	81.930792
3	Home Score	2.104919
4	MatchId	622.527183
5	OfficialId	1.056650
6	OppPossessionSecs	inf
7	OppScore	inf
8	Player Id	1.012881
9	PositionId	1.853186
10	PossessionSecs	Inf
11	PTB Tackle Result	1.196111
12	Set Type	3.117679
13	Total Involved Tacklers	1.071384

14	Raw Tackle Number	15.490458
15	RoundId	625.068105
16	RunOn	1.647206
17	Score	inf
18	SeasonId	0.000000
19	SeqNumber	329.357503
20	Set	82.178051
21	Tackle	14.228941
22	TotalPossessionSecs	inf
23	ZonePhysical	1.002946
24	ZonePossession	3.732256
25	GameTime	86.912625
26	ElapsedTime	68.128301
27	IsHome	1.103638
28	CurrentMargin	inf

Figure 5.12 VIF results for all features

Index	Feature	Variance Inflation Factor (VIF)
0	Away Score	2.011938
1	DurationSecs	1.253010
3	Home Score	2.104919
5	OfficialId	1.056650
8	Player Id	1.012881
9	PositionId	1.853186
11	PTB Tackle Result	1.196111
12	Set Type	3.117679
13	Total Involved Tacklers	1.071384
16	RunOn	1.647206
18	SeasonId	0.000000

23	ZonePhysical	1.002946
24	ZonePossession	3.732256
27	IsHome	1.103638

Figure 5.13 VIF results after removal of high VIF features

Model	MSE _{train}	MSE _{test}	RMSE _{train}	RMSE _{test}	R ² _{train}	R ² _{test}	AIC	BIC
Linear Regression	0.4027	0.4109	0.6346	0.6410	0.1404	0.1357	160023.3717	160405.7345
Ridge Regression	0.4027	0.4109	0.6346	0.6410	0.1404	0.1357	160023.3717	160405.7345
LASSO Regression	0.4551	0.4629	0.6746	0.6804	0.0285	0.0263	160023.3717	160405.7345
ElasticNet Regression	0.4246	0.4323	0.6516	0.6575	0.0936	0.0907	160023.3717	160405.7345
XGBoost Regression	0.3872	0.3986	0.6223	0.6133	0.1734	0.1617	-	-

Figure 5.14 Summary of select Models without Interaction terms

Model	MSE _{train}	MSE _{test}	RMSE _{train}	RMSE _{test}	R ² _{train}	R ² _{test}	AIC	BIC
Linear Regression	0.4300	0.4418	0.6557	0.6647	0.1255	0.1233	165407.5763	165566.1169
Ridge Regression	0.4300	0.4418	0.6557	0.6647	0.1255	0.1233	165407.5763	165566.1169
LASSO Regression	0.4811	0.4936	0.6936	0.7026	0.0215	0.0206	165407.5763	165566.1169
ElasticNet Regression	0.4480	0.4604	0.6693	0.6785	0.0889	0.0865	165407.5763	165566.1169
XGBoost Regression	0.4162	0.4320	0.6451	0.6573	0.1535	0.1428	-	-

Figure 5.15 Summary of select Models with Interaction terms

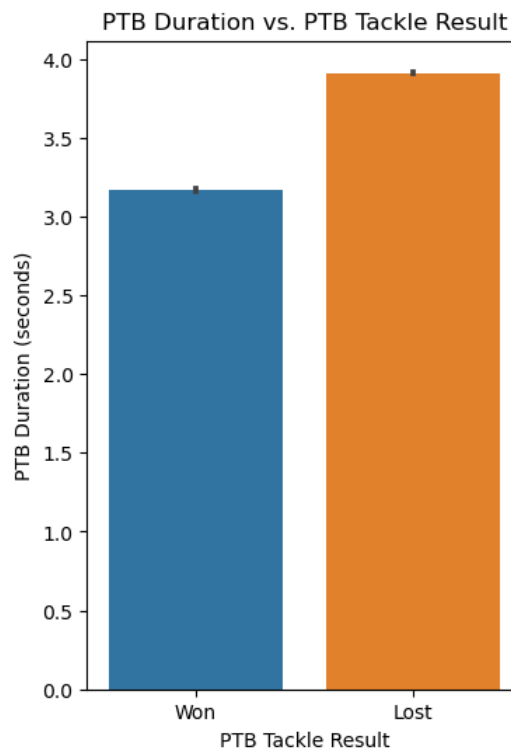
Interaction Term	p-value
ElapsedTime * CurrentMargin	1.536940e-05

Total Involved Tacklers > 2 * PTBTackleResult	4.34553e-30
SetType * PTBTackle Result	5.993324e-09

Figure 5.16 p-values of Interaction Terms

Feature	LASSO Coefficient
PTB Tackle Result	-0.271179
Set Type	0.025810
Tackle	-0.037922

Figure 5.17 Output of LASSO coefficients



Barplot of PTB Durations and PTB Tackle Result

Model	MSE _{train}	MSE _{test}	RMSE _{train}	RMSE _{test}	R ² _{train}	R ² _{test}	AIC	BIC
Linear Regression	0.4088	0.4159	0.6394	0.6449	0.127 3	0.125 2	161217.681 7	161348.244 5
Ridge Regression	0.4088	0.4159	0.6394	0.6449	0.127 3	0.125 2	161217.681 7	161348.244 5
LASSO Regression	0.4589	0.4664	0.6774	0.6829	0.020 3	0.019 1	161217.681 7	161348.244 5
ElasticNet	0.4261	0.4334	0.6528	0.6584	0.090	0.088	161217.681	161348.244

Regression					4	4	7	5
XGBoost Regression	0.3953	0.4063	0.6287	0.6374	0.1562	0.1455	–	–
Random Forest	0.0587	0.4287	0.2424	0.6548	0.8746	0.0983	–	–

Figure 5.21 OLS Summary of All Models with Feature Selection

Model	(Train)	(Test)	(Train)	(Test)	(Train)	R ² (Test)	AIC	BIC	Remarks
Linear Regression	0.408803	0.415930	0.639377	0.644926	0.127338	0.125192	161217.681656	161348.244549	Baseline model. Similar results to Ridge Regression.
Ridge Regression	0.408803	0.415930	0.639377	0.644926	0.127338	0.125192	161217.681656	161348.244549	Chosen for optimization due to handling multicollinearity effectively.
LASSO Regression	0.458903	0.466365	0.677424	0.682909	0.020391	0.019114	161217.681656	161348.244549	Underperformed with higher MSE and lower R ² values than baseline.
ElasticNet Regression	0.426085	0.433437	0.652752	0.658359	0.090447	0.088370	161217.681656	161348.244549	Worse performance compared to Linear and Ridge Regression.
XGBoost Regression	0.395280	0.406285	0.628713	0.637405	0.156205	0.145478	N/A	N/A	Best overall model with lowest MSE and highest R ² values.

Summary of Initial Models

Ridge Regression

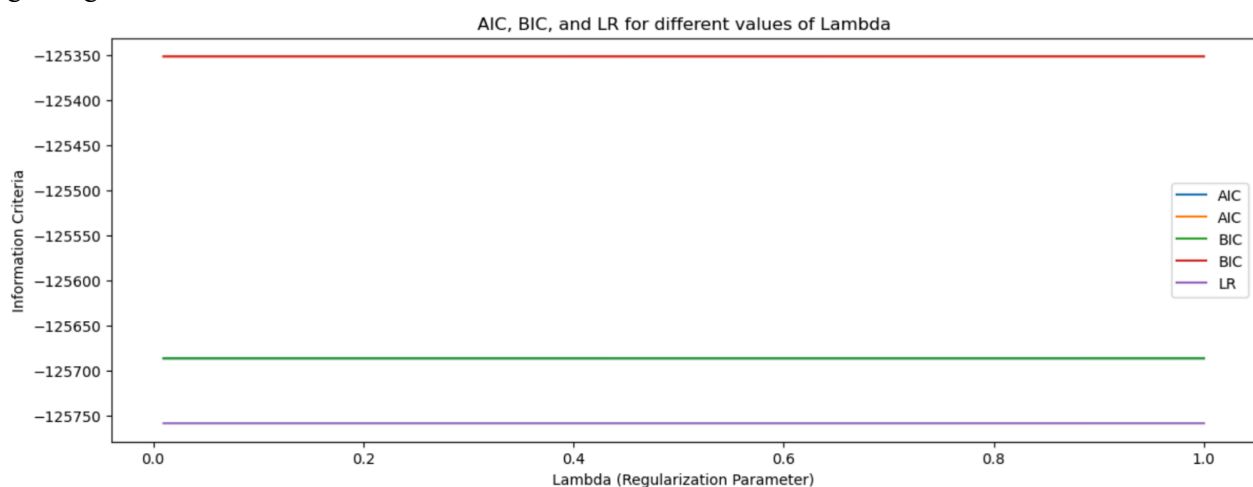


Figure 5.22 Effect of Different Lambda (λ) values in Ridge Regression

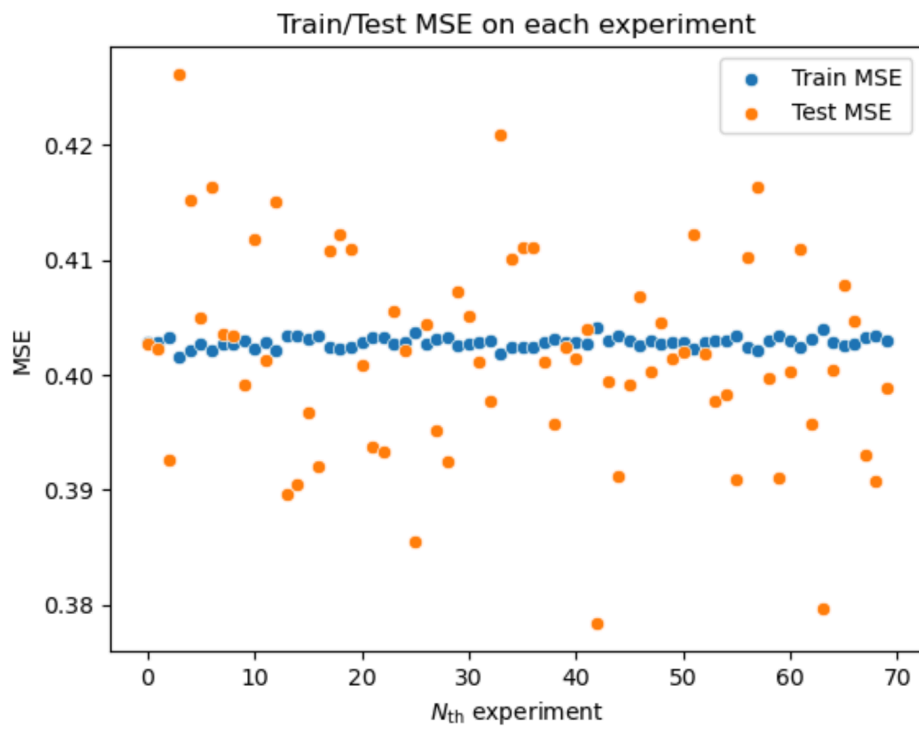
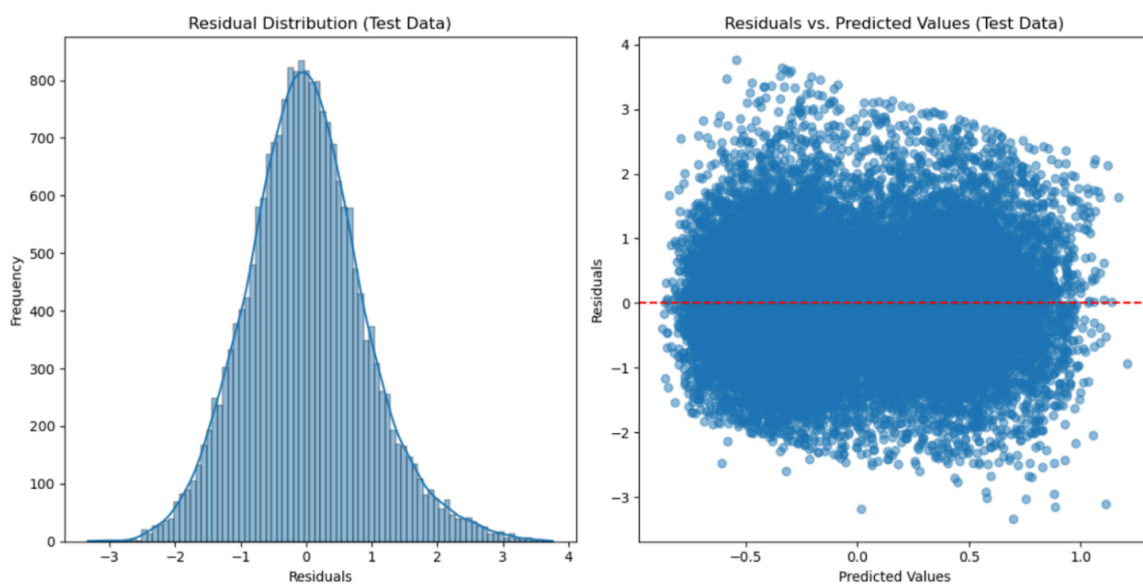


Figure 5.23 Train and Test MSE of Ridge Regression over 70 runs



Ridge Regression

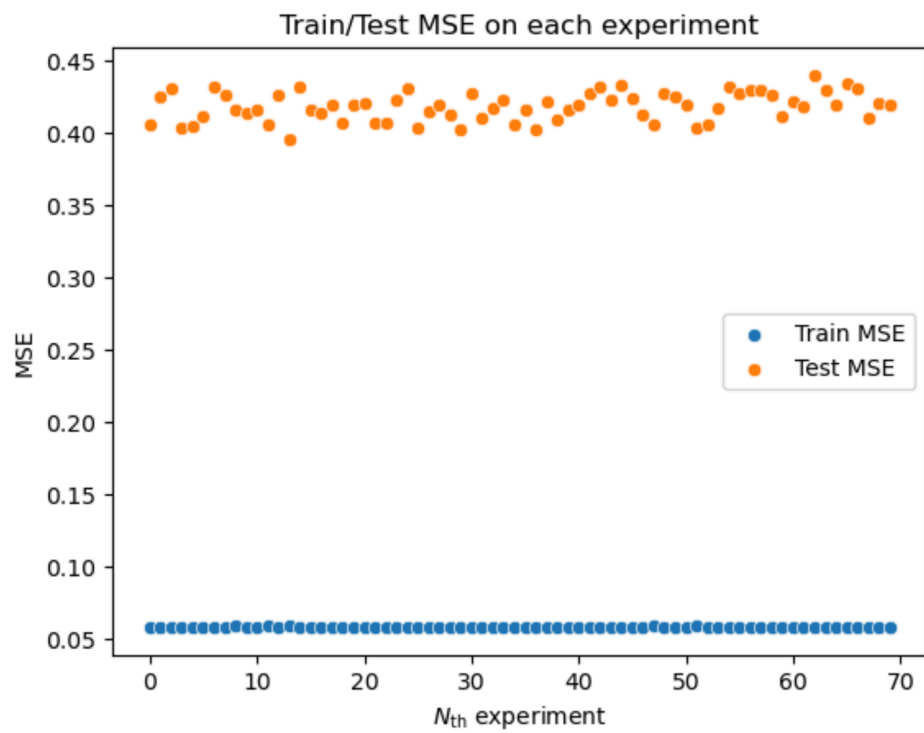


Figure 5.27 Train and Test MSE of Random Forest over 70 runs

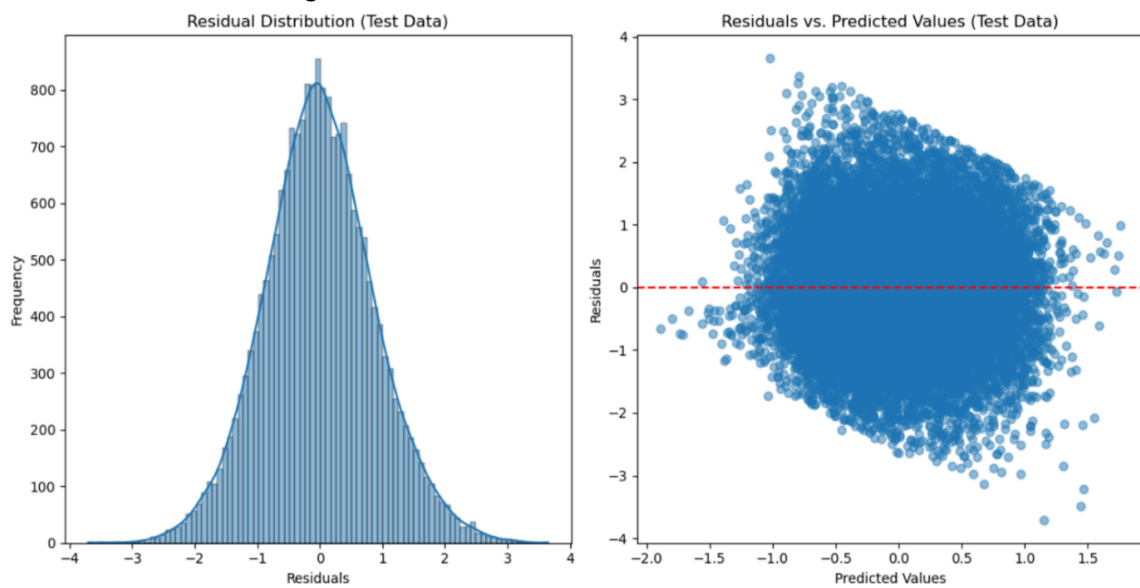


Figure 5.28 Residuals of Random Forest

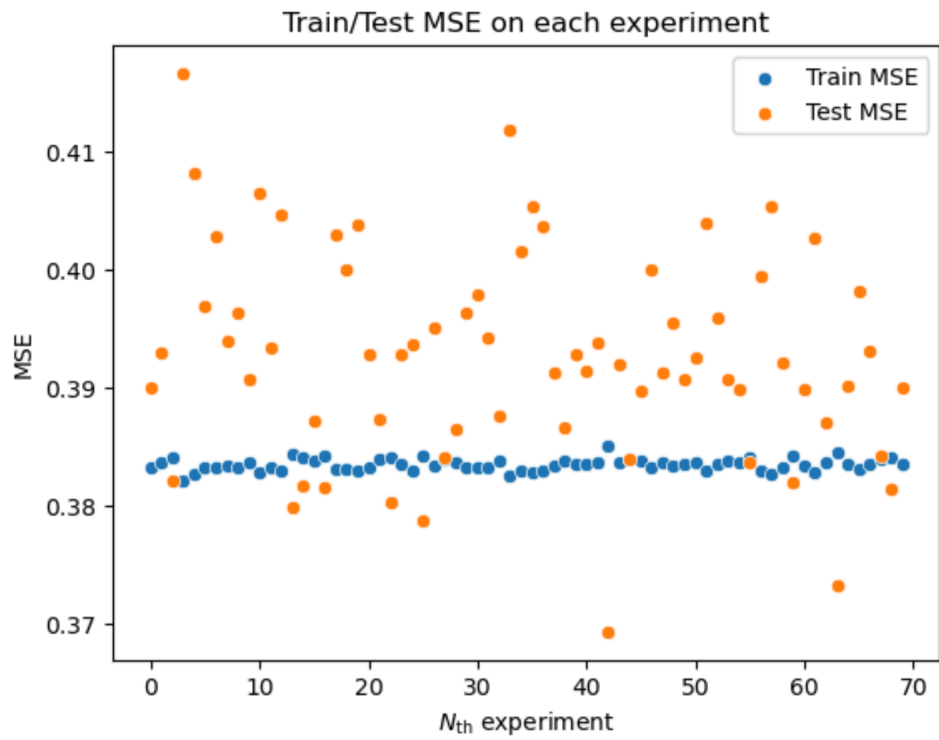


Figure 5.25 Train and Test MSE of XGboost over 70 runs

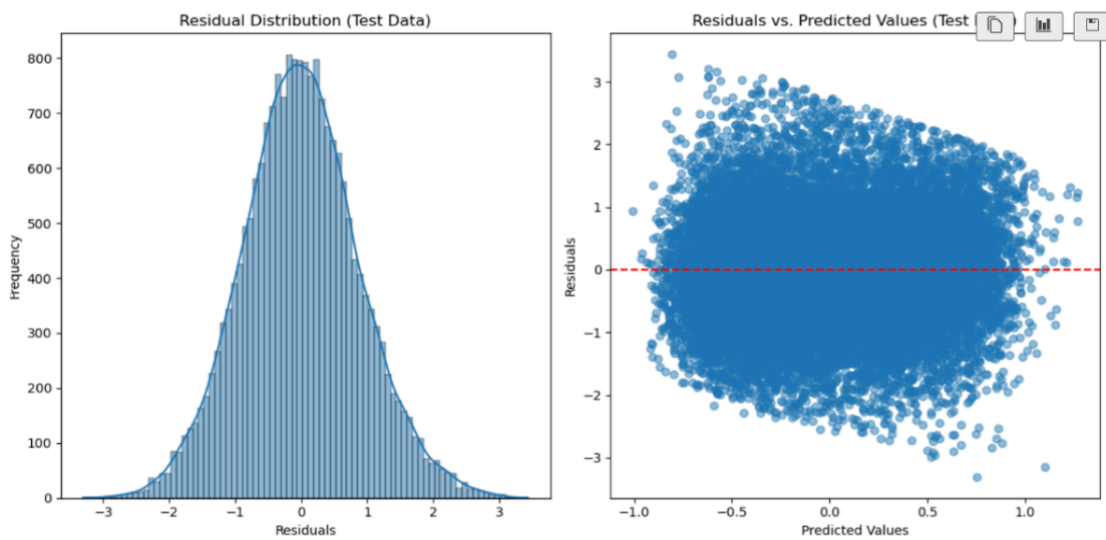


Figure 5.26 Residuals of XGBoost

Model	Train MSE	Test MSE	Train R ²	Test R ²	Advantages	Disadvantages
Ridge Regression	~0.409	~0.416	Moderate (~0.63)	Moderate (~0.64)	Handles multicollinearity well, robust against overfitting, consistent performance across splits.	Limited ability to model complex non-linear relationships.
XGBoost	~0.386	~0.405	Low (~0.18)	Moderate (~0.15)	Captures complex patterns, robust against overfitting, low bias, consistent performance.	More complex to tune and implement compared to simpler models.
Random Forest	~0.058	~0.431	High (~0.87)	Low (~0.10)	Captures non-linear relationships well, strong fit to training data.	Severe overfitting, poor generalization to unseen data.

Summary of model results

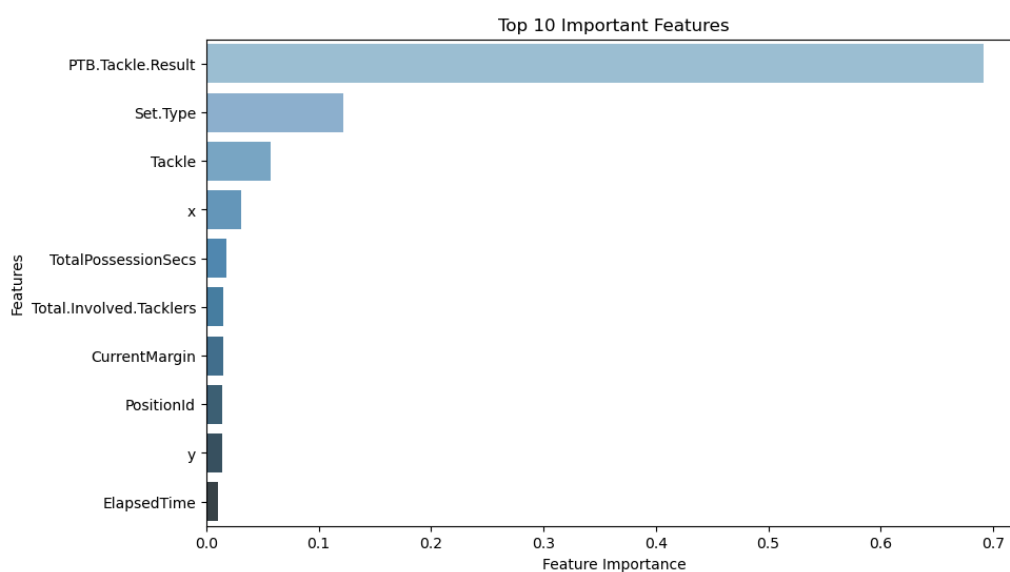


Figure 5.26 Feature Importance Ranked by XGBoost Coefficients

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import scipy.stats as stats
import xgboost as xgb
```

```

from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from statsmodels.regression.linear_model import OLS
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV
from scipy.stats import zscore

# Read data and give basic descriptive statistic information

ptb = pd.read_csv("ptb_data.csv")
ptb.head()
ptb.info()
ptb.describe(include = 'all')

# Find missing data

missing_values = ptb.isnull().sum()
missing_columns = missing_values[missing_values > 0]
print(missing_columns)

# Since PTB Defence has too much missing data, we remove column altogether along with missing
data

plt.figure(figsize=(8, 6))
sns.scatterplot(x='PTB Defence', y='DurationSecs', data=ptb)
plt.title('Scatter Plot of PTB Defence vs DurationSecs')
plt.show()

correlation = ptb['PTB Defence'].corr(ptb['DurationSecs'])
print("Correlation between PTB Defence and DurationSecs:", correlation)

# Given the weak correlation and the fact we lose close to 80% of our data if we just remove
missing data -> get rid of PTB Defense then drop rest of missing data of 32 rows

ptb_remove = ptb.drop(['PTB Defence'], axis=1)

# Drop all rows with missing data (69 rows gone)

ptb_remove = ptb_remove.dropna()

# Correlation matrix

corr_matrix = ptb.corr()

```

```

plt.figure(figsize=(11, 11))
sns.heatmap(corr_matrix, annot=True,annot_kws={"size": 5}, cmap='coolwarm', square=True,
fmt=".2f", linewidths = 0.5)
plt.title("Correlation Matrix")
plt.show()

# Histogram

ptb_remove.hist(figsize=(12, 8), bins=20)
plt.tight_layout()
plt.show()

# Boxplots

for column in ptb_remove.select_dtypes(include=['float', 'int']).columns:
    plt.figure(figsize=(6, 4))
    sns.boxplot(y=ptb_remove[column])
    plt.title(f"{column}")
    plt.show()

# Looking at histogram for duration secs it is not good with mean of 3.52 but with very large
values -> lots of outliers.
# So get rid of outliers using Q1/Q3 +- 1.5 iqr

Q1 = ptb['DurationSecs'].quantile(0.25)
Q3 = ptb['DurationSecs'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
ptb_clean = ptb_remove[(ptb_remove['DurationSecs'] >= lower_bound) &
(ptb_remove['DurationSecs'] <= upper_bound)]

# Variables to remove that are useless

corr_matrix = ptb_clean.corr()
plt.figure(figsize=(11, 11))
sns.heatmap(corr_matrix, annot=True,annot_kws={"size": 5}, cmap='coolwarm', square=True,
fmt=".2f", linewidths = 0.5)
plt.title("Correlation Matrix")
plt.show()

# PossessionSecs and OppPossessionSecs
# Set and OppPossessionSecs (Essential)
# Set and PossessionSecs(Essential)
# TotalPossessionSecs and OppPossessionSecs
# TotalPossessionSecs and PossessionSecs
# Tackle and Raw Tackle Number
# TotalPossessionSecs and Set(Essential)

```

```

# ElapsedTime and GameTime
# Have High correaltion but some are essential

# Boxplot

ptb_tackle_result_col = 'PTB Tackle Result'
ptb_duration_col = 'DurationSecs'
barplotdata = ptb_clean.copy()
barplotdata['PTB Tackle Result'] = barplotdata['PTB Tackle Result'].map({0: 'Lost', 1: 'Won'})

plt.figure(figsize=(4, 6))
sns.barplot(data=barplotdata, x=ptb_tackle_result_col, y=ptb_duration_col)
plt.title("PTB Duration vs. PTB Tackle Result")
plt.xlabel("PTB Tackle Result")
plt.ylabel("PTB Duration (seconds)")
plt.show()

# Scatterplots

plt.figure(figsize=(8, 6))
plt.scatter(ptb_clean['OppPossessionSecs'], ptb_clean['PossessionSecs'])
plt.xlabel('OppPossessionSecs')
plt.ylabel('PossessionSecs')
plt.title('Scatterplot of OppPossessionSecs vs PossessionSecs')
plt.show()

plt.figure(figsize=(8, 6))
plt.scatter(ptb_clean['TotalPossessionSecs'], ptb_clean['PossessionSecs'])
plt.xlabel('TotalPossessionSecs')
plt.ylabel('PossessionSecs')
plt.title('Scatterplot of TotalPossessionSecs vs PossessionSecs')
plt.show()

plt.figure(figsize=(8, 6))
plt.scatter(ptb_clean['TotalPossessionSecs'], ptb_clean['OppPossessionSecs'])
plt.xlabel('TotalPossessionSecs')
plt.ylabel('OppPossessionSecs')
plt.title('Scatterplot of TotalPossessionSecs vs OppPossessionSecs')
plt.show()

plt.figure(figsize=(8, 6))
plt.scatter(ptb_clean['Tackle'], ptb_clean['Raw Tackle Number'])
plt.xlabel('Tackle')
plt.ylabel('Raw Tackle Number')
plt.title('Scatterplot of Tackle vs Raw Tackle Number')
plt.show()

plt.figure(figsize=(8, 6))

```

```

plt.scatter(ptb_clean['ElapsedTime'], ptb_clean['GameTime'])
plt.xlabel('ElapsedTime')
plt.ylabel('GameTime')
plt.title('Scatterplot of ElapsedTime vs GameTime')
plt.show()

# Get VIF

df_vif = ptb_clean.drop(columns=['EventName'], axis=1)

# Convert columns to numeric, setting invalid parsing as NaN
df_numeric = df_vif.apply(pd.to_numeric, errors='coerce')

# Check if any columns have NaN values after coercion
non_numeric_columns = df_numeric.columns[df_numeric.isna().any()].tolist()
print("Non-numeric columns or columns with NaN values:", non_numeric_columns)

# Drop non-numeric columns
df_cleaned = df_vif.drop(columns=non_numeric_columns)

# Function to calculate VIFs
def calculate_vif(df):
    vif_data = pd.DataFrame()
#numeric only features
    vif_data["feature"] = df.columns
    vif_data["VIF"] = [variance_inflation_factor(df.values, i) for i in range(df.shape[1])]
    return vif_data
vif_results = calculate_vif(df_cleaned).round(4)
print(vif_results)

vif_under_10 = vif_results[vif_results["VIF"] <= 10]
print(vif_under_10)

# MODELLING

# Prepare data

ptb_clean['x'] = ((ptb_clean['ZonePossession'] - 1) // 7) + 1 # x axis(1-12)
ptb_clean['y'] = ((ptb_clean['ZonePossession'] - 1) % 7) + 1 # y axis (1-7)
ptb_drop = ptb_clean.drop(columns=['Anonymize 1PlayerId', 'Half', 'PTB Ultimate Outcome', 'Player
Id', 'SeasonId', 'RoundId', 'Raw Tackle Number', 'Club Id', 'Opposition
Id', 'ZonePossession', 'Score', 'ZonePhysical', 'Set', 'EventName', 'Home Score', 'OppScore', 'PTB
Contest', 'Away Score', 'OppPossessionSecs', 'MatchId', 'OfficialId', 'PossessionSecs',
'GameTime', 'SeqNumber'])

duration_heatmap = ptb_clean.groupby(['y', 'x'])['DurationSecs'].mean().unstack(fill_value=0)

plt.figure(figsize=(10, 6))

```

```

sns.heatmap(duration_heatmap, annot=True, fmt=".2f", cmap="YlGnBu", cbar=True,
            xticklabels=duration_heatmap.columns, yticklabels=duration_heatmap.index)
plt.title("Heatmap of PTB between Average and Duration Across Zones")
plt.xlabel("Columns (x)")
plt.ylabel("Rows (y)")
plt.gca().invert_yaxis()
plt.show()

# Make new variables for x and y of zones for possession of the ball

Q1 = ptb_drop['CurrentMargin'].quantile(0.25)
Q3 = ptb_drop['CurrentMargin'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

ptb_final = ptb_drop[(ptb_drop['CurrentMargin'] >= lower_bound) & (ptb_drop['CurrentMargin']
<= upper_bound)]

# Standardization

scaler = StandardScaler()

# Get columns to be standardized
columns_to_scale = ['DurationSecs', 'ElapsedTime']

ptb_final[columns_to_scale] = scaler.fit_transform(ptb_final[columns_to_scale]).round(4)

print(ptb_final[columns_to_scale].head())

# Create dummies for categorical columns then convert rest to ints

ptb_final = pd.get_dummies(ptb_final, drop_first=True)
ptb_final = ptb_final.astype(int)

# Remove some columns

ptb_final.describe()

ptb_final =
ptb_final.drop(columns=['WeatherConditionName_Showers', 'WeatherConditionName_Snow', 'WeatherCon
ditionName_Unknown'])

X = ptb_final.drop(columns=['DurationSecs'])
y = ptb_final['DurationSecs']

# Simple OLS model

```

```

X = sm.add_constant(X)

# Fit model
model = sm.OLS(y, X).fit()

# Delete variables with high p values
alpha = 0.05
while True:
    # Get the variable with the largest p value
    p_values = model.pvalues
    max_p_value = p_values.max()

    # Stop if all p-values are less than alpha
    if max_p_value < alpha:
        break

    # Find the variable corresponding to the largest p-value
    max_p_variable = p_values.idxmax()
    print(f"Removing {max_p_variable} with p-value {max_p_value}")

    # Remove the variable and refit the model
    X = X.drop(columns=[max_p_variable])
    model = sm.OLS(y, X).fit()

# Final model results
print("Final Model Summary:")
print(model.summary())
print("P-values for all variables in the model:")
print(model.pvalues)

# For comparison of linear regression models

# Divide training set and test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Model evaluation indicator function
def evaluate_model(model, X_train, X_test, y_train, y_test):

    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)

    # MSE and RMSE
    mse_train = mean_squared_error(y_train, y_pred_train)
    mse_test = mean_squared_error(y_test, y_pred_test)
    rmse_train = np.sqrt(mse_train)
    rmse_test = np.sqrt(mse_test)

```

```

# R2
r2_train = r2_score(y_train, y_pred_train)
r2_test = r2_score(y_test, y_pred_test)

return mse_train, mse_test, rmse_train, rmse_test, r2_train, r2_test

# Assessment using AIC and BIC
def calculate_aic_bic(model, X_train, y_train):
    X_train_sm = sm.add_constant(X_train) # 添加常数项
    ols_model = sm.OLS(y_train, X_train_sm).fit()
    aic = ols_model.aic
    bic = ols_model.bic
    return aic, bic

# Initialize models
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=0.01, random_state=42),
    'Lasso Regression': Lasso(alpha=0.1),
    'ElasticNet Regression': ElasticNet(alpha=0.1, l1_ratio=0.5),
    'XGBoost Regression': XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=3,
random_state=42)
}

# Record assessment results
results = []

# Iterate over each model, train and evaluate
for name, model in models.items():
    model.fit(X_train, y_train)

    # MSE, RMSE, R2
    mse_train, mse_test, rmse_train, rmse_test, r2_train, r2_test = evaluate_model(model,
X_train, X_test, y_train, y_test)

    # Calculate AIC and BIC
    if name != 'XGBoost Regression':
        aic, bic = calculate_aic_bic(model, X_train, y_train)
    else:
        aic, bic = np.nan, np.nan # XGBoost 不支持 AIC 和 BIC

    results.append({
        'Model': name,
        'MSE (Train)': mse_train,
        'MSE (Test)': mse_test,
        'RMSE (Train)': rmse_train,
        'RMSE (Test)': rmse_test,
        'R² (Train)': r2_train,

```

```

        'R² (Test)': r2_test,
        'AIC': aic,
        'BIC': bic
    })

# Convert the results to a DataFrame for viewing
results_df = pd.DataFrame(results).round(4)
print(results_df)

# Optimization

def calculate_aic_bic(model, X_train, y_train):
    # Add constant term (intercept) to X_train
    X_train_sm = sm.add_constant(X_train)

    # Fit the OLS model
    ols_model = sm.OLS(y_train, X_train_sm).fit()

    # Extract AIC and BIC
    aic = ols_model.aic
    bic = ols_model.bic

    return aic, bic

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Expand lambda range from 0.01 to 50
lambdas = np.linspace(0.01, 50, 100)
aic_values = []
bic_values = []
lr_values = []

# Perform ridge regression for each lambda value
for alpha in lambdas:
    ridge_model = Ridge(alpha=alpha)
    ridge_model.fit(X_train, y_train)

    # Calculate predicted value
    y_pred_train = ridge_model.predict(X_train)

    # Calculate AIC and BIC
    aic, bic = calculate_aic_bic(ridge_model, X_train, y_train)

    # Calculate the likelihood ratio (LR) using the mean squared error
    mse_train = mean_squared_error(y_train, y_pred_train)
    lr = len(y_train) * np.log(mse_train) # Approximation

    # Append the values

```

```

aic_values.append(aic)
bic_values.append(bic)
lr_values.append(lr)

# Find the value of lambda that minimizes AIC, BIC, and LR
optimal_lambda_aic = lambdas[np.argmin(aic_values)]
optimal_lambda_bic = lambdas[np.argmin(bic_values)]
optimal_lambda_lr = lambdas[np.argmin(lr_values)]

print("Optimal Lambda based on AIC:", optimal_lambda_aic)
print("Optimal Lambda based on BIC:", optimal_lambda_bic)
print("Optimal Lambda based on LR:", optimal_lambda_lr)

# Visualize the results
plt.figure(figsize=(14, 5))
plt.plot(lambdas, aic_values, label='AIC')
plt.plot(lambdas, bic_values, label='BIC')
plt.plot(lambdas, lr_values, label='LR')
plt.xlabel('Lambda (Regularization Parameter)')
plt.ylabel('Information Criteria')
plt.title('AIC, BIC, and LR for different values of Lambda')
plt.legend()

# Best Ridge

scaler = StandardScaler()

columns_to_scale = ['ElapsedTime']

ptb_final[columns_to_scale] = scaler.fit_transform(ptb_final[columns_to_scale])
ptb_final = pd.get_dummies(ptb_final, drop_first=True)
ptb_final = ptb_final.astype(int)

# Define the alpha parameter range to search for
param_grid = {'alpha': [
0.60,0.61,0.62,0.63,0.64,0.65,0.66,0.67,0.68,0.69,0.70,0.71,0.72,0.73,0.74,0.75,0.76,0.77,0.78
,0.79,0.80]}

# Initialize the ridge regression model
ridge = Ridge()

# Parameter tuning using GridSearchCV
grid_search = GridSearchCV(ridge, param_grid, cv=5, scoring='neg_mean_squared_error')
grid_search.fit(X_train, y_train)

# Get the best alpha value
best_alpha = grid_search.best_params_['alpha']
print(f"Best Alpha for Ridge: {best_alpha}")

```

```

# Use optimal alpha to train the model
best_ridge_model = Ridge(alpha=best_alpha)
best_ridge_model.fit(X_train, y_train)
y_pred_ridge = best_ridge_model.predict(X_test)

# Evaluate the adjusted ridge regression model
mse_ridge = mean_squared_error(y_test, y_pred_ridge).round(4)
r2_ridge = r2_score(y_test, y_pred_ridge).round(4)

print(f"Best Ridge Regression MSE: {mse_ridge}")
print(f"Best Ridge Regression R²: {r2_ridge}")

# Random forest

rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_train_rf = rf_model.predict(X_train)
y_pred_test_rf = rf_model.predict(X_test)

print("Random Forest:")
print("Train MSE:", round(mean_squared_error(y_train, y_pred_train_rf), 4))
print("Test MSE:", round(mean_squared_error(y_test, y_pred_test_rf), 4))
print("Train R²:", round(r2_score(y_train, y_pred_train_rf), 4))
print("Test R²:", round(r2_score(y_test, y_pred_test_rf), 4))

# Xgb regression

# Initialize and train the XGBoost model using optimal parameters
xgb_model = RandomForestRegressor(n_estimators=100, random_state=42)
xgb_model.fit(X_train, y_train)

y_train_pred = xgb_model.predict(X_train)
y_test_pred = xgb_model.predict(X_test)

train_residuals = y_train - y_train_pred
test_residuals = y_test - y_test_pred

plt.figure(figsize=(12, 6))

# Test set residual distribution
plt.subplot(1, 2, 1)
sns.histplot(test_residuals, kde=True)
plt.title("Residual Distribution (Test Data)")
plt.xlabel("Residuals")
plt.ylabel("Frequency")

# Residuals and predicted values of test data

```

```

plt.subplot(1, 2, 2)
plt.scatter(y_test_pred, test_residuals, alpha=0.5)
plt.axhline(y=0, color='r', linestyle='--')
plt.title("Residuals vs. Predicted Values (Test Data)")
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")

plt.tight_layout()
plt.show()

# Output evaluation indicators
mse = mean_squared_error(y_test, y_test_pred).round(4)
r2 = r2_score(y_test, y_test_pred).round(4)
print("XGBoost Regression:")
print(f"Test MSE: {mse}")
print(f"Test R2: {r2}")

# Create XGBoost model
xgb_model = XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
xgb_model.fit(X_train, y_train)

# Predict training set and test set
y_train_pred = xgb_model.predict(X_train)
y_test_pred = xgb_model.predict(X_test)

train_mse = mean_squared_error(y_train, y_train_pred).round(4)
test_mse = mean_squared_error(y_test, y_test_pred).round(4)
train_r2 = r2_score(y_train, y_train_pred).round(4)
test_r2 = r2_score(y_test, y_test_pred).round(4)

print("XGBoost Regression:")
print("Train MSE:", train_mse)
print("Test MSE:", test_mse)
print("Train R2:", train_r2)
print("Test R2:", test_r2)

"""-----"""
feature_importances = xgb_model.feature_importances_
features = X_train.columns # Adjust if you're using a different dataset

# Create a DataFrame with features and their importance scores
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importances
})

# Sort by importance and select the top 10 features
top_10_features = importance_df.sort_values(by='Importance', ascending=False).head(10)

```

```

# Plot using seaborn for better aesthetics
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=top_10_features, palette='Blues_d')
plt.title('Top 10 Important Features')
plt.xlabel('Feature Importance')
plt.ylabel('Features')
plt.show()

# MSE for xgb

# Set model parameters
model = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=1.0,
    min_child_weight=5,
    random_state=42
)

# Perform multiple random data splits and calculate MSE
num_experiments = 70
train_mse_list = []
test_mse_list = []

for i in range(num_experiments):
    # Split data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.05, random_state=i)

    # Train the model
    model.fit(X_train, y_train)

    # Predict and calculate MSE
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)

    train_mse = mean_squared_error(y_train, y_pred_train)
    test_mse = mean_squared_error(y_test, y_pred_test)

    train_mse_list.append(train_mse)
    test_mse_list.append(test_mse)

# Calculate average and standard deviation of MSE
train_mse_avg = np.mean(train_mse_list).round(4)
test_mse_avg = np.mean(test_mse_list).round(4)
train_mse_std = np.std(train_mse_list).round(4)

```

```

test_mse_std = np.std(test_mse_list).round(4)

# Print results
print("Average Train MSE:", train_mse_avg)
print("Average Test MSE:", test_mse_avg)
print("Train MSE Std Dev:", train_mse_std)
print("Test MSE Std Dev:", test_mse_std)

# Scatter plot of Train/Test MSE across experiments
sns.scatterplot(x=range(num_experiments), y=train_mse_list, label='Train MSE')
sns.scatterplot(x=range(num_experiments), y=test_mse_list, label='Test MSE')
plt.title('Train/Test MSE on each experiment')
plt.xlabel(r'$N_{\mathrm{th}}$ experiment')
plt.ylabel('MSE')
plt.legend()
plt.show()

# Final training with the full training set and evaluation
X_train_final, X_test_final, y_train_final, y_test_final = train_test_split(X, y,
test_size=0.05, random_state=42)
model.fit(X_train_final, y_train_final)
y_pred_train_final = model.predict(X_train_final)
y_pred_test_final = model.predict(X_test_final)

final_train_mse = mean_squared_error(y_train_final, y_pred_train_final).round(4)
final_test_mse = mean_squared_error(y_test_final, y_pred_test_final).round(4)

print("Final Train MSE:", final_train_mse)
print("Final Test MSE:", final_test_mse)

# Optimization of xgb

# Parameter grid for RandomizedSearchCV
param_dist = {
    'n_estimators': [100, 200, 500],
    'learning_rate': [0.01, 0.05, 0.1, 0.3],
    'max_depth': [3, 5, 7, 10],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'min_child_weight': [1, 3, 5]
}

# Perform randomized search with cross-validation
xgb_model = xgb.XGBRegressor(objective="reg:squarederror", random_state=42)
random_search = RandomizedSearchCV(xgb_model, param_distributions=param_dist,
scoring='neg_mean_squared_error',
                                   n_iter=10, cv=5, random_state=42, n_jobs=-1)
random_search.fit(X_train, y_train)

```

```

best_params = random_search.best_params_

# Use DMatrix for training and early stopping
dtrain = xgb.DMatrix(X_train, label=y_train)
dtest = xgb.DMatrix(X_test, label=y_test)

# Define parameters with the best found values
params = {
    'objective': 'reg:squarederror',
    'eval_metric': 'rmse',
    'n_estimators': best_params['n_estimators'],
    'learning_rate': best_params['learning_rate'],
    'max_depth': best_params['max_depth'],
    'subsample': best_params['subsample'],
    'colsample_bytree': best_params['colsample_bytree'],
    'min_child_weight': best_params['min_child_weight']
}

# Train with early stopping
evals = [(dtrain, 'train'), (dtest, 'eval')]
model = xgb.train(params, dtrain, num_boost_round=500, evals=evals, early_stopping_rounds=10,
verbose_eval=False)

# Predictions and evaluation
y_train_pred = model.predict(dtrain)
y_test_pred = model.predict(dtest)

# Calculate MSE and R²
train_mse = mean_squared_error(y_train, y_train_pred).round(4)
test_mse = mean_squared_error(y_test, y_test_pred).round(4)
train_r2 = r2_score(y_train, y_train_pred).round(4)
test_r2 = r2_score(y_test, y_test_pred).round(4)

# Output results
print("XGBoost Regression:")
print("Train MSE:", train_mse)
print("Test MSE:", test_mse)
print("Train R²:", train_r2)
print("Test R²:", test_r2)
print("Best Parameters:", best_params)

```